# Sieci Neuronowe
# Laboratorium 04
## Algorytm wstecznej propagacji błędu dla wielowarstwowej w pełni połączonej sieci neuronowej

Andrzej Kordecki

Zakład Teorii Maszyn i Robotów
Instytut Techniki Lotniczej i Mechaniki Stosowanej
Wydział Mechaniczny Energetyki i Lotnictwa
Politechnika Warszawska
2026

## Wstęp

Celem ćwiczenia jest zapoznanie się z jedną z najważniejszych i najczęściej stosowanych metod uczenia sieci neuronowych — algorytmem wstecznej propagacji błędu (backpropagation). Metoda ta stanowi podstawę działania większości współczesnych modeli uczenia maszynowego, umożliwiając efektywne dostosowywanie wag neuronów w procesie treningu.

Laboratorium to jest kontynuacją poprzednich zajęć i wymaga posiadania wcześniej zaimplementowanych w języku Python funkcji umożliwiających tworzenie oraz obsługę wielowarstwowej sieci neuronowej (MLP – Multi-Layer Perceptron). W trakcie ćwiczenia studenci zaimplementują procedurę uczenia sieci z wykorzystaniem propagacji wstecznej, a następnie przetestują jej działanie na wybranych danych.

## Algorytm wstecznej propagacji błędu

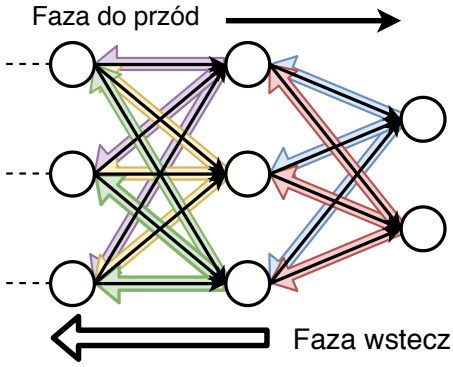
Algorytm wstecznej propagacji błędów, opracowany w połowie lat osiemdziesiątych XX wieku, był kamieniem milowym w nauczaniu wielowarstwowych sieci neuronowych. Działanie algorytmu opiera się o wykorzystanie metody gradientu prostego do wyznaczenia optymalnych parametrów sieci, którymi podczas laboratoriów będą wagi określające siłę połączenia pomiędzy neuronami. Szkolenie w algorytmie wstecznej propagacji błędu przebiega w dwóch fazach (Rys. 1):

1. faza w przód - wagi sieci są stałe, a sygnał funkcyjny od wejścia jest propagowany przez sieć, warstwa po warstwie, aż do wyznaczenia wyjścia NN.

2. faza wstecz - wyznacza błąd sieci neuronowej poprzez porównanie wyjścia sieci z pożądaną jej odpowiedzią. Wynikowy sygnał błędu jest następnie propagowany przez sieć, ponownie warstwa po warstwie, ale propagacja błędu odbywa się w kierunku wstecznym od wyjścia do wejścia sieci. W tej fazie dokonywane są kolejne aktualizacje wag NN.

Pierwsza faza w przód jest wymagane do wyznaczenia wyjścia sieci i na ile jest ono różne zgodnie z przyjętą funkcje strat od pożądanego wyjście sieci określonego przez dane uczące. Wyznaczony w ten sposób błąd stanowi podstawę do aktualizacji parametrów sieci. Należy zauważyć, że różne neurony miały różny wpływ na wyjście sieci, co powoduje że wielkością aktualizacji poszczególnych parametrów powinna być inna i zależna od ich wpływu. Metodą pozwalająca na jego określenie jest metoda gradientowa prostego zastosowana do optymalizacji parametrów sieci.

Przejdźmy na zapis matematyczny. Uczenie wielowarstwowej sieci w pełni połączonej wymaga dostosowania wag $w_{ij}(n)$ w celu zminimalizowania funkcji strat $E(w)$ zależnej od pożądanych wartości wyjściowych $t$ oraz




Rysunek 1: Przepływ sygnału ramach algorytmu wstecznej propagacji błędu

wartości wyjściowych ostatniej $N$ warstwy sieci $y$ dla $p$ próbki danych:

$$E(w) = E(\sum_p \sum_j \epsilon(t_{j,p}, y_j^{(N)}(x_p, w)))$$

Aktualizacja wag wykorzystuje metodą gradientu prostego bezpośrednio zastosowaną na sieci neuronowej:

$$w_{h,g}^{new} = w_{h,g}^{old} - \eta \frac{\partial E}{\partial w_{h,g}}|_{w_{h,g}=w_{h,g}^{old}}$$

- Tylko wyjścia $y_j^{(N)}$ ostatniej warstwy występują w funkcji,
- Sygnał wyjściowy z ostatniej warstwy zależy od wszystkich ustawionych wag.

Stąd, istotnym elementem algorytmu optymalizacji wag sieci jest błąd obliczany w ostatniej warstwie sieci i następnie propagowany do pierwszych warstw sieci. Błąd $\delta$ zależy od wpływu poszczególnego neurony na wynik działania sieci. Działanie algorytmu wstecznej propagacji błędu można zapisać za pomocą następujących równań:

- dla warstwy wyjściowej (N):
$$\delta_g^{(N)} = E'(y_g) f'(\sum_j y_j^{(N-1)} w_{j,g}^{(N)})$$
- dla warstwy ukrytych (1 do N-1):
$$\delta_g^{(n)} = (\sum_i \delta_i^{(n+1)} w_{g,i}^{(n+1)}) f'(\sum_j y_j^{(n-1)} w_{j,g}^{(n)})$$

Równanie aktualizacji wag:

$$\Delta w_{h,g}^{(n)} = \eta \sum_p \delta_g^{(n)} y_h^{(n-1)}$$

Zgodnie z powyższymi zależnościami możemy zapisać za pomocą gradientu zależnego od pochodnej funkcji strat, rozważanego neuronu i potencjału neuronu:

$$\frac{\partial E(w)}{\partial w} = \frac{\partial E(y)}{\partial y} \frac{\partial y(v)}{\partial v} \frac{\partial v(w)}{\partial w}$$

Równanie aktualizacji wymaga znania pochodnych funkcji aktywacji neuronu i funkcji strat. Stąd, jednym naszym zadaniem podczas dzisiejszego laboratoriów będzie dodanie opcji wyznaczenia gradientu wielu funkcji programu oraz nowych wartości pośrednich wykorzystywanych w obliczenia.

## Pochodne funkcji aktywacji

Poniżej zostaną opisane pochodne funkcje aktywacji wykorzystywane podczas laboratoriów. Podane wzory stanowią uzupełnienie wzorów podanych podczas poprzednich laboratoriów.

**Liniowa funkcja aktywacji (Rys. 2).** Równanie wyjścia funkcji:

$$f(x) = purelin(x) = x$$

Pochodna funkcji aktywacji:

$$\frac{\partial f(x)}{dx} = 1$$

**Rectified linear unit (ReLU) (Rys. 3).** Równanie wyjścia funkcji:

$$f(x) = relu(x) = \max(0, v)$$

Funkcja aktywacji ReLU nie jest różniczkowalna w 0, czyli że pochodna nie jest zdefiniowana (lub nie istnieje). Zazwyczaj przyjmujemy wartość pochodnych w pobliżu nieciągłości: 0, 1 lub 0,5.

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \\ niezdefiniowana & x = 0 \end{cases}$$

W praktyce stosunkowo rzadko zdarza się, aby x=0 (dokładnie) w kontekście głębokiego uczenia się.

**Sigmoid - funkcja logistyczna (Rys. 4).** Równanie wyjścia funkcji:

$$f(x) = sigmf(x) = \frac{1}{1 + e^{-x}}$$

Pochodna funkcji aktywacji:

$$\frac{\partial f(x)}{dx} = f(x)(1 - f(x))$$

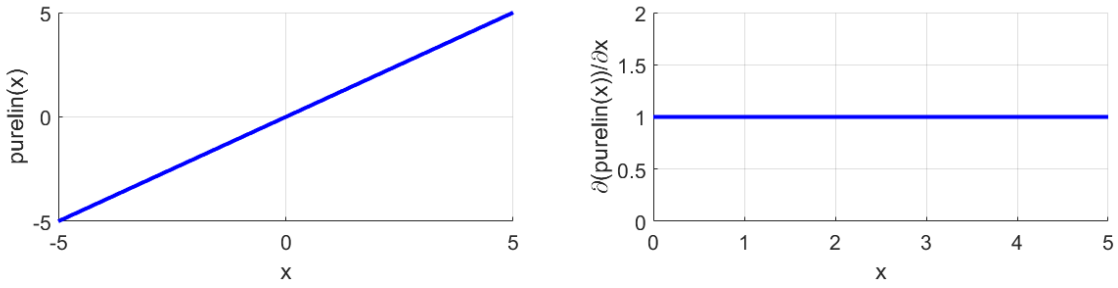
**Funkcja tangensu hiperbolicznego.** Równanie wyjścia funkcji:

$$tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

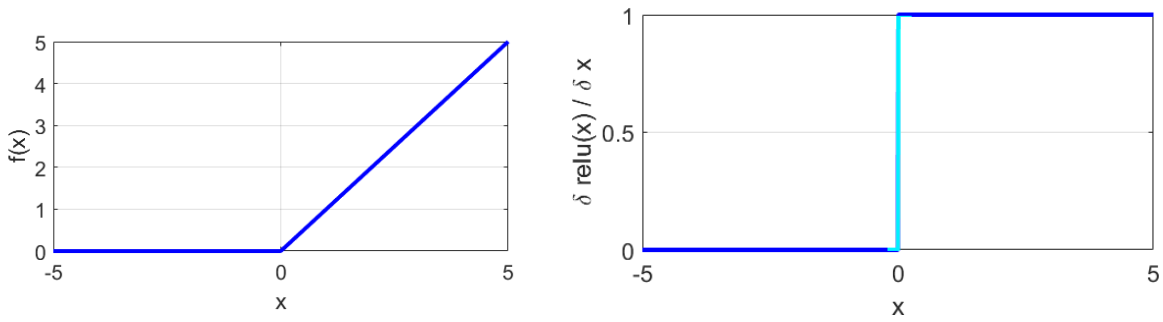
Pochodna funkcji aktywacji:

$$\frac{\partial f(x)}{dx} = 1 - f^2(x)$$

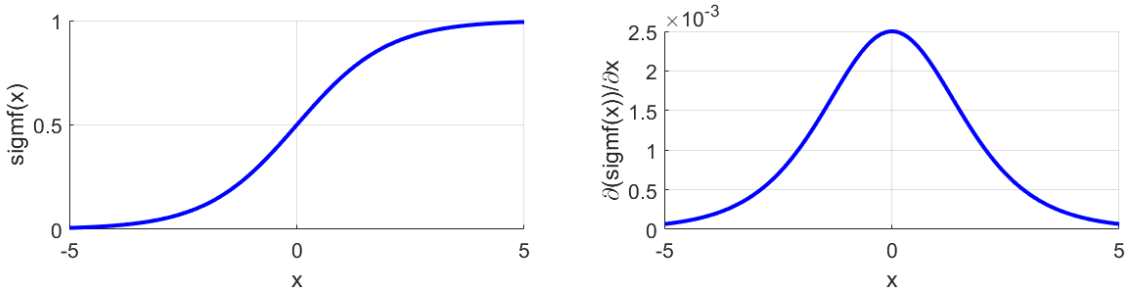
Przebieg funkcji wyjścia i pochodnej jest zbliżony do funkcji logistycznej.




Rysunek 2: Wartość wyjściowa oraz pochodna liniowej funkcji aktywacji




Rysunek 3: Wartość wyjściowa oraz pochodna funkcji aktywacji ReLu




Rysunek 4: Wartość wyjściowa oraz pochodna logistycznej funkcji aktywacji

## Pochodne funkcji strat

Poniżej zostaną opisane pochodne funkcje strat wykorzystywane podczas laboratoriów. Podane wzory stanowią uzupełnienie wzorów podanych podczas poprzednich laboratoriów.

**Funkcji strat binarnej entropii krzyżowej** jest powszechnie stosowana w zadaniach klasyfikacji binarnej, gdzie model ma za zadanie przewidzieć jedną z dwóch klas. Definiuje się ją jako miarę różnicy pomiędzy przewidywanymi prawdopodobieństwami a rzeczywistymi etykietami klas. Równanie funkcji strat:

$$L = -\sum_{i=1}^{N}[t_i \log(y_i) + (1 - t_i)\log(1 - y_i]$$

Gdzie $t_i$ to rzeczywista, która wynosi 1 dla pozytywnej i 0 dla negatywnej próbki danych, a $y_i$ określa wyjście sieci w zakresie 0-1. Oznacza to, że zastosowanie tej funkcji strat do porwanego działania wymaga zastosowania w ostatniej warstwie funkcji aktywacji neurony o wyjściu w zakresie 0-1. Pochodna entropii krzyżowej stosowane w algorytmie wstecznej propagacji błędu dla pojedyńczej próbki:

$$\frac{\partial L}{\partial t} = -(\frac{t}{y} - \frac{1-t}{1-y})$$

**Funkcji strat MSE.** Błąd średniokwadratowy to miara często stosowana uczeniu maszynowym do oceny wydajności modeli regresyjnych. Błąd średniokwadratowy jest wrażliwy na odstające wartości, ponieważ różnice są podnoszone do kwadratu, co powoduje, że większe błędy mają znacznie większy wpływ na końcowy wynik. Równanie funkcji strat:

$$L = \frac{1}{N}\sum_{i=1}^{N}(t_i - y_i)^2$$

Gdzie: $t_i$ to rzeczywista wartość próbki, a $y_i$ to wyjście sieci neuronowej dla tej próbki, N - to liczba próbek w zbiorze danych. Pochodna MSE stosowane w algorytmie wstecznej propagacji błędu dla pojedyńczej próbki:

$$\frac{\partial L}{\partial t} = (y_i - t_i)$$

## Przykład propagacji sygnału błędu wstecz

Przykład ten stanowi rozwinięcie przykładu z poprzedniego laboratorium o propagacje sygnału wstecz i aktualizacje wag sieci neuronowej. W celu łatwiejszego zrozumienia wykonywanych operacji przykład zostanie opisany od początku wraz z elementami z poprzedniej instrukcji. Propagacja sygnału w przód będzie przeprowadzona dla sieci dwuwarstwowej jednokierunkowej (Rys. 5).

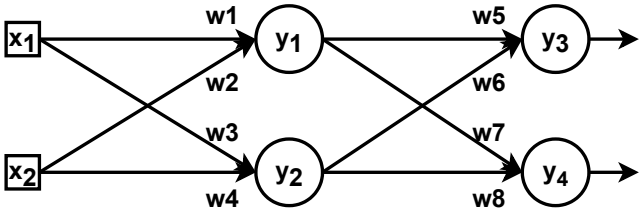
Wartości wyjściowe każdego neuronu można obliczyć w następujący sposób:

$$y_1 = f_1(v_1) = f_1(w_1 * x_1 + w_2 * x_2) \quad y_2 = f_2(w_3 * x_1 + w_4 * x_2)$$

$$y_3 = f_3(v_3) = f_3(w_5 * y_1 + w_6 * y_2) \quad y_4 = f_4(w_7 * y_1 + w_8 * y_2)$$

Równania te możemy zapisać w postaci macierzowej:

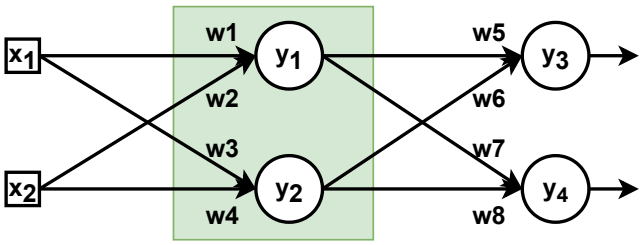
$$X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad W_1 = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} \quad W_2 = \begin{bmatrix} w_5 & w_6 \\ w_7 & w_8 \end{bmatrix}$$




Rysunek 5: Sieć dwuwarstwowa jednokierunkowa

$$V_1 = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}, \quad V_2 = \begin{bmatrix} v_3 \\ v_4 \end{bmatrix},$$
$$Y_1 = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}, \quad Y_2 = \begin{bmatrix} y_3 \\ y_4 \end{bmatrix}.$$

Potencjał wszystkich neuronów pierwszej warstwy można obliczyć w następujący sposób (Rys. 6):




Rysunek 6: Obliczenia w fazie w przód związane z pierwszą warstwą sieci neuronowej

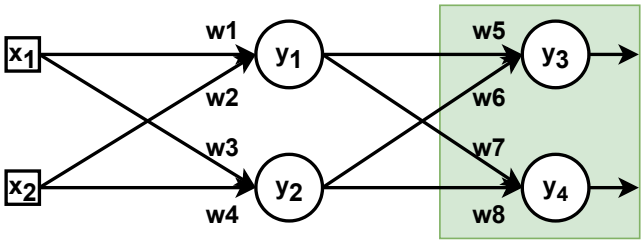
$$V_1 = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = W_1 * X = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Wartości wyjściowe neuronów w pierwszej warstwie sieci neuronowej:

$$Y_1 = F_1(W_1 * X)$$

Wartości wyjściowe, ostatniej warstwy sieci neuronowej obliczane są na podstawie wartości wyjściowych neuronów w warstwie pierwszej sieci (Rys. 7):

$$Y_2 = F_2(W_2 * Y_1)$$



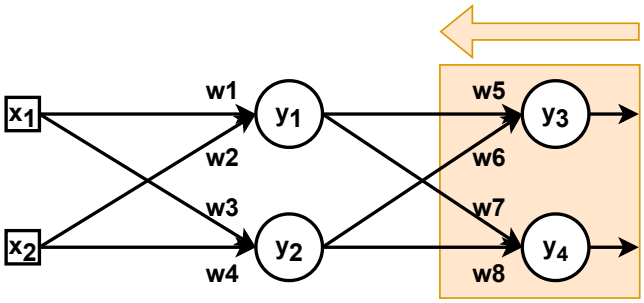

Rysunek 7: Obliczenia związane w fazie w przód z drugą, wyjściową warstwą sieci neuronowej

Obydwa równania możemy zapisać w jednym wzorze:

$$Y_{i+1} = F_{i+1}(W_{i+1} * Y_i), \text{ dla } Y_0 = X, i = 0, ..., N-1$$

gdzie $N$ to liczba warstw sieci neuronowej.

Otrzymując sygnał wyjściowy z ostatniej warstwy sieci otrzymujemy wartość predykcji naszej sieci dla danej próbki danych. Umożliwia to wyznaczenia wartości błędu predykcji sieci $Y$ poprzez porównanie otrzymanej wartości z pożądaną $T$ za pomocą założonej funkcji strat $E$. Naszym zadaniem w fazie wstecz ustalenie "błędu"predykcji dla każdego neuronu rozpoczynając od ostatniej warstwy wyjściowej $N = 2$ (Rys. 8).



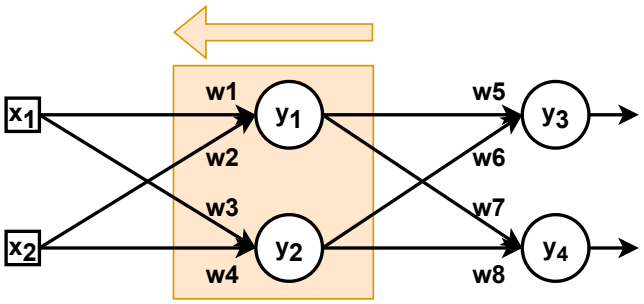

Rysunek 8: Obliczenia w fazie wstecz związane z drugą, wyjściową warstwą sieci neuronowej

$$\delta^{(N)} = E'(Y_N) \odot F'_N(Y_{N-1} * W_N)$$

Błąd $\delta_g^{(N)}$ musi mieć ten sam rozmiar co oba pomnożone wektory gradientu. Dlatego w tym przypadku stosujemy mnożenie przez poszczególne elementy odpowiadających sobie macierzy.

$$\delta^{(2)} = E'(Y_2) \odot F'_2(Y_1 * W_2)$$

Algorytm propagacji wstecznej dla warstw ukrytych $1, \ldots, N-1$, który dla przykładu ogranicza się do $n = 1$ (Rys. 9):




Rysunek 9: Obliczenia w fazie wstecz związane z pierwszą, ukrytą warstwą sieci neuronowej

$$\delta^{(n)} = (\delta^{(n+1)} * W_{n+1}) \odot F'_n(Y_{n-1} * W_N)$$

Wyznaczona $(\delta^{(n+1)} * W_{n+1})$ będzie wektorem o rozmiarze $n$.

Ostatnim krokiem jest aktualizacja wag dla warstw ukrytych $1, \ldots, N$:

$$\Delta W_n = \eta \delta^{(n)} * Y_{n-1}$$

- Parametr $\eta$ jest wartością skalarną i jest jednym z hiperparametrów sieci (ustawianych przez użytkownika).
- Wynik mnożenia wektorów wejściowych o rozmiarze $q$ i wektora delta o rozmiarze $n$ powinien być macierzą o rozmiarze $q \times n$.
- Musimy pamiętać, że rozmiar macierzy wag pierwszej warstwy wynosi $k \times n$ i dla każdej wagi $w_i$ dodajemy współczynnik $\delta_i^{(n)} * y_i^{(n-1)}$.

# Zadanie 1

Zaimplementuj wyznaczanie wartości i pochodnych dla funkcji aktywacji i funkcji strat:

- Funkcji strat: binarnej entropii krzyżowej oraz błędu średnio-kwadratowego.
- Funkcji aktywacji: liniowej, logistycznej, tangensu hiperbolicznego i ReLu.

Pamiętaj, że można wykorzystać kod programu z laboratorium 02 i 03. Jeżeli nie wykonano tego zadania podczas wcześniejszych laboratoriów to możemy pożyczyć kod z poprzednich laboratoriów od innego uczestnika zajęć. Dodaj na początku programu funkcję wpływającą na generację danych losowych *np.random.seed*(). Przy jej pisaniu można posłużyć się poniższym kodem.

```
import sys

np.random.seed(42)

class Loss_fcn:

    def __init__(self):
        self.functions = {
            'mse': self.mse,
            'binary_cross_entropy': self.binary_cross_entropy
        }

    # Loss/error value calculated for all input data sample
    def output(self, name, expected, outputs, derivative):
        if name in self.functions:
            return self.functions[name](expected, outputs, derivative)
        else:
            sys.exit(f"Error: Loss function '{name}' not found.")

    # Uzupelnij brakujacy kod programu


class Activation_fcn:

    def __init__(self):
        self.functions = {
            'linear': self.linear,
            'sigmoid': self.logistic,
            'logistic': self.logistic,
            'tanh': self.tanh,
            'relu': self.relu
        }

    # Claculate neuron output
    def output(self, layer, name, derivative=False):
    if name in self.functions:
      return self.functions[name](layer['activation_potential'], layer['output'], derivative)
    else:
      sys.exit(f"Error: Activation function '{name}' not found.")

    # Uzupelnij brakujacy kod programu
```

# Zadanie 2

Zaimplementuj algorytm wstecznej propagacji błędów do treningu sieci neuronowej. W tym dodaj odpowiednie funkcje do klasy "Neural_network"odpowiedzialną za trening sieci neuronowej. Struktura nowych funkcji jest już zarysowana, ale brakuje istotnych elementów kodu programu oznaczonych jako "None"oraz odpowiedniego połączenia kodu w całość. Realizacja zadania wymaga wykorzystania kodu z poprzednich laboratoriów (np. funkcji "forward_propagate"z laboratorium 03). Poniżej przedstawiono strukturę nowych funkcji:

- Funkcja "train"to główna funkcja odpowiedzialna za trening sieci neuronowej, której kod został w pełni udostępniony. W ramach funkcji wywoływane są wszystkie najważniejsze procesy algorytmu wstecznej propagacji błędu: propagacja sygnału w przód ("forward_propagate"), wyznaczanie błędu funkcji strat ("sum_error"), propagacja sygnału błędu wstecz ("backward_propagate") oraz aktualizacja wag ("update_weights").

Trening odbywa się na wszystkich próbkach zbioru uczącego ("x_train", "y_train"), a proces ten jest zamknięty w pętli w ramach wykonywania jednej epoki obliczeń. Liczba epok treningu określona w ramach drugiej pętli jest przez parametr "n_epoch". Dodatkowo w ramach głównej funkcji "train"przekazywane są dwa parametry "loss_function"określająca typ funkcji strat (zgodnie z poprzednim zadaniem instrukcji) oraz stała uczenia "l_rate"wykorzystywane przy aktualizacji wag.

```
    # Train a network for a fixed number of epochs
    def train(self, nnetwork, x_train, y_train, l_rate=0.01, n_epoch=100, \
                                            loss_function='mse'):
        for epoch in range(n_epoch):
            sum_error = 0
            for iter, (x_row, y_row) in enumerate(zip(x_train, y_train)):

                self.forward_propagate(nnetwork, x_row)

                loss = loss_fcn()
                sum_error = np.sum(loss.loss(loss_function, y_row,
                                nnetwork[-1]['output'], derivative=False))

                self.backward_propagate(loss_function, nnetwork, y_row)

                self.update_weights(nnetwork, x_row, l_rate)

            print('>epoch=%d, loss=%.3f' % (epoch + 1, sum_error))

        return nnetwork
```

- Funkcja "backward_propagate"to funkcja odpowiedzialna za propagacje sygnału błędu od końca sieci, przez wszystkie jej warstwy, aż do początku sieci. Wyznaczamy w ramach funkcji błąd (oznaczony w wzorach przez symbol $\delta$, a w kodzie $nnetwork[i]['delta']$) związany jest z każdym neuronem sieci neuronowej o strukturze zdefiniowanej w $nnetwork$. Należy zwrócić uwagę, że metody wyznaczania błędu są rożne dla ostatniej warstwy sieci i wcześniejszych warstw sieci (dlatego warunek w kodzie $if\ i < N$). Funkcja musi w pierwszej kolejności obliczyć błąd neuronów ostatniej warstwy $N$, który jest określony przez gradient funkcji strat. Wcześniejsze warstwy sieci błąd wyznaczają w oparciu o błąd $nnetwork[i+1]['delta']$, czyli kolejnej warstwy sieci (pamiętamy, że propagujemy sygnał błędu wstecz) wyskalowany o wagi związane z połącznikiem z tą warstwą sieci. Ostatecznie wyznaczamy błąd $nnetwork[i]['delta']$ dla wszystkich warstwy sieci przez wyskalowanie przez pochodną funkcji aktywacji neuronów związanych z daną warstwą sieci. Warto zapoznać się z działaniem funkcji "np.matmul()".

```
    def backward_propagate(self, loss_function, nnetwork, expected):
        N = len(nnetwork)-1
        for i in range(N, 0, -1):
            errors = []
            # Uzupelnij brakujacy kod programu
            if i<N:
                weights = None
                errors = None
            else:
                errors = None

            nnetwork[i]['delta'] = np.multiply(errors, \
                self.af.output(nnetwork[i], nnetwork[i]['activation_function'], derivative=
    True))
```

- funkcja "update_weights"funkcja aktualizuje wagi neuronu na zgodnie z wartością błędu neuronu $nnetwork[i]['delta']$, wartością sygnału wejściowego neuronu związanego z tą wagą $inputs$ oraz stałej uczenia $l_rate$. Warto zapoznać się z działaniem funkcji "np.matmul()".

```
    def update_weights(self, nnetwork, inputs, l_rate):
        inp = inputs
        for i in range(1, len(nnetwork)):
            # Uzupelnij brakujacy kod programu
            nnetwork[i]['weights'] -= None
            inp = nnetwork[i]['output']
```

## Zadanie 3

Przetestuj działanie funkcji na przykładzie zadania regresji i klasyfikacji

- Przykład regresji

```
def generate_regression_data(n=30):
    # Generate regression dataset
    X = np.linspace(-5, 5, n).reshape(-1, 1)
    y = np.sin(2 * X) + np.cos(X) + 5
    # simulate noise
    data_noise = np.random.normal(0, 0.2, n).reshape(-1, 1)
    # Generate training data
    Y = y + data_noise

    return X.reshape(-1, 1), Y.reshape(-1, 1)

def test_regression():
    # Read data
    X, Y = generate_regression_data()

    # Create network
    model = Neural_network()
    structure = [{'type': 'input', 'units': 1},
                 {'type': 'dense', 'units': 8, 'activation_function': 'tanh'},
                 {'type': 'dense', 'units': 8, 'activation_function': 'tanh'},
                 {'type': 'dense', 'units': 1, 'activation_function': 'linear'}]

    network = model.create_network(structure)

    model.train(network, X, Y, 0.01, 4000, 'mse', 0)

    predicted = model.predict(network, X)
    std = np.std(predicted - Y)
    print("\nStandard deviation = {}".format(std))

    X_test = np.linspace(-7, 7, 100).reshape(-1, 1)
    X_test = np.array(X_test).tolist()
    predicted = model.predict(network, X_test)

    plt.plot(X, Y, 'r--o', label="Training data")
    plt.plot(X_test, predicted, 'b--x', label="Predicted")
    plt.legend()
    plt.grid()
    plt.show()
```

- Przykład klasyfikacji

```
def generate_classification_data(n=30):
    # Class 1 - samples generation
    X1_1 = 1 + 4 * np.random.rand(n, 1)
    X1_2 = 1 + 4 * np.random.rand(n, 1)
    class1 = np.concatenate((X1_1, X1_2), axis=1)
    Y1 = np.ones(n)

    # Class 0 - samples generation
    X0_1 = 3 + 4 * np.random.rand(n, 1)
    X0_2 = 3 + 4 * np.random.rand(n, 1)
    class0 = np.concatenate((X0_1, X0_2), axis=1)
    Y0 = np.zeros(n)

    X = np.concatenate((class1, class0))
    Y = np.concatenate((Y1, Y0))

    idx0 = [i for i, v in enumerate(Y) if v == 0]
    idx1 = [i for i, v in enumerate(Y) if v == 1]

    return X, Y, idx0, idx1

def test_classification():
    # Read data
    X, Y, idx0, idx1 = generate_classification_data()

    # Create network
    model = Neural_network()
    structure = [{'type': 'input', 'units': 2},
                 {'type': 'dense', 'units': 4, 'activation_function': 'relu'},
                 {'type': 'dense', 'units': 4, 'activation_function': 'relu'},
```

```
                {'type': 'dense', 'units': 1, 'activation_function': 'logistic'}]

network = model.create_network(structure)

model.train(network, X, Y, 0.0001, 2000, 'binary_cross_entropy', 0)

y = model.predict(network, X)
t = 0
for n, m in zip(y, Y):
    t += 1 - np.abs(np.round(np.array(n)) - np.array(m))
    print(f"pred = {n}, pred_round = {np.round(n)}, true = {m}")

ACC = t / len(X)
print(f"\nClassification accuracy = {ACC * 100}%")

# Plotting decision regions
xx, yy = np.meshgrid(np.arange(0, 8, 0.1),
                     np.arange(0, 8, 0.1))

X_vis = np.c_[xx.ravel(), yy.ravel()]

h = model.predict(network, X_vis)
h = np.array(h) >= 0.5
h = np.reshape(h, (len(xx), len(yy)))

plt.contourf(xx, yy, h, cmap='jet')
plt.scatter(X[idx1, 0], X[idx1, 1], marker='^', c="red", edgecolors="white", label="
class 1")
plt.scatter(X[idx0, 0], X[idx0, 1], marker='o', c="blue", edgecolors="white", label="
class 0")
plt.show()
```

Jeżeli wystąpią problemy z otrzymaniem dobrego wyniku treningu:

- Sprawdź, czy implementacja nie zawiera błędów matematycznych poprzez sprawdzenie, czy ogólny trend zmian zmniejsza funkcje strat z epoki na epokę.
- Zmień ustawinia sieci neuronowej: zmiana współczynnika uczenia $l_rate$, zwiększenie liczby próbek generowanego zadania $n$, czy powtórzenie uczenia dla innej inicjalizacji wag przez zmianę generowanych wartości losowych ustawionych w *np.random.seed*().

Dobierz parametry sieci, które polepszą wynik klasyfikacji i regresji. Jakie ogólne zalecenie możesz wskazać, które pozwolą poprawić najszybciej wyniki sieci?